

Introduction to Databases & SQL

with DuckDB

OMIS 105 — Day 1 · Your journey into data begins here

🗺️ What We'll Cover Today

Part 1 — The Big Picture

- What is a database?
- Why not just use Excel?
- Meet DuckDB 🦆

Part 2 — Your First SQL

- Anatomy of a table
- SELECT — the workhorse
- Filtering with WHERE
- Sorting with ORDER BY

Part 3 — Aggregation

- Summarizing with GROUP BY
- COUNT, SUM, AVG, MIN, MAX

Part 4 — Joining Tables

- What is a JOIN?
- INNER JOIN in action

Hands-on Lab 🧪

- Run it yourself in Jupyter!

SQL: Language of Data

Hands-on Lab 🧪

- Run it yourself in Jupyter!

> 💡 *No prior experience needed.*
>
> *If you can use Google Sheets, you can learn SQL.*

Part 1

The Big Picture

🤔 What Is a Database?

A **database** is an organized collection of structured data, stored electronically, designed to be **queried and managed efficiently**.

Think of it as a **super-powered, organized filing cabinet** that can answer questions instantly — even with millions of records.

Filing Cabinet 📂	Database 🗄️
Physical folders	Tables
Documents in a folder	Rows
Fields on a form	Columns
Finding files manually	SQL Query
One person at a time	Many users simultaneously

😫 Why Not Just Use Excel or CSV?

You might be thinking: **"I already have Excel, why learn this?"**

Excel struggles with:

- Files > 1 million rows
- Multiple people editing at once
- Complex relationships between data
- Repeatable, automated analysis
- Data integrity & validation
- Joining multiple datasets

Databases excel at:

- **Billions** of rows, no sweat
- Concurrent multi-user access
- Foreign keys & relationships
- Version-controlled queries
- Data types & constraints
- Lightning-fast JOINs

🔥 ***Real talk:*** *The average company database has tens of millions of rows. Excel would crash. DuckDB handles it in seconds.*

🦆 Meet DuckDB — The Database That Runs Anywhere

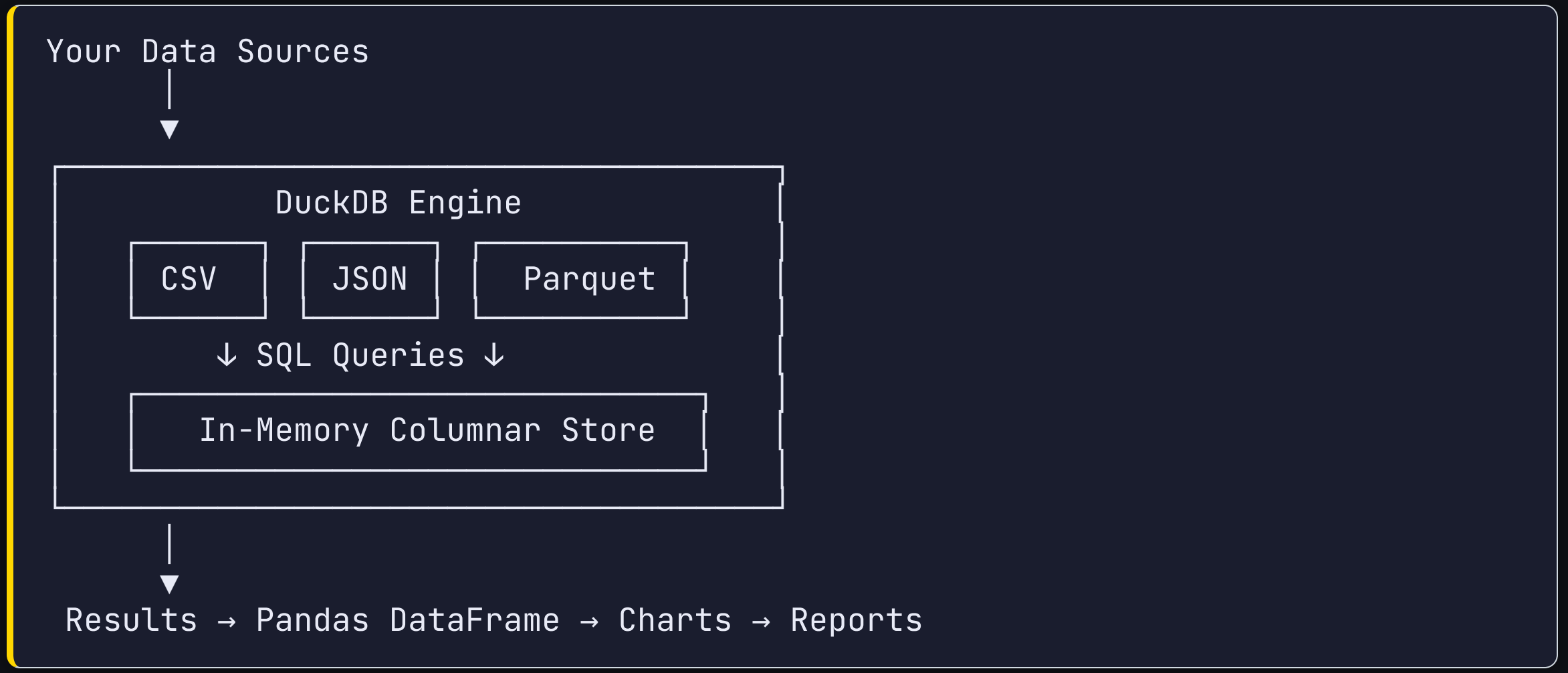
DuckDB is a modern, open-source analytical database that runs **entirely inside your Python environment** — no server, no installation headaches.

```
import duckdb          # That's it. You're ready!
```

Why DuckDB is perfect for learning:

- 🚀 **Zero setup** — install with `pip install "duckdb ⩾ 2.0"`
- 🐍 **Python-native** — works perfectly in Jupyter notebooks
- ⚡ **Blazing fast** — column-oriented storage engine
- 📂 **Reads anything** — CSV, Parquet, JSON, Excel, even Pandas DataFrames

How DuckDB Fits In the Data World



DuckDB slots neatly into any Python data pipeline.

Part 2

Your First SQL

🏛️ Anatomy of a Table

A database table looks a lot like a spreadsheet — but with **strict rules**. Every column has a **data type**, and every row is a **record**.

```
-- Our sample table: students
```

student_id	name	age	major	gpa
1	Alice Chen	20	Computer Science	3.9
2	Bob Martin	22	Mathematics	3.4
3	Carol White	21	Computer Science	3.7
4	David Lee	23	Physics	3.2
5	Emma Davis	20	Mathematics	3.8

Column data types: `INTEGER`, `VARCHAR`, `DECIMAL`, `DATE`, `BOOLEAN` ...

📖 SELECT — The Workhorse of SQL

`SELECT` retrieves data from a table. It's the command you'll use **90% of the time**.

Syntax:

```
SELECT column1, column2, ...
FROM table_name;
```

Examples:

```
-- Get all columns (the * means "everything")
SELECT * FROM students;

-- Get only specific columns
SELECT name, gpa FROM students;

-- Add a computed column
SELECT name, gpa, gpa * 4.0 AS scaled_score FROM students;
```

🔍 WHERE — Filtering Your Data

`WHERE` lets you ask **"give me only rows that match this condition"**.

```sql
SELECT column1, column2
FROM table_name
WHERE condition;
```

Comparison operators:

Operator	Meaning	Example
`=`	Equal to	`major = 'Computer Science'`
`≠` or `<>`	Not equal	`major ≠ 'Physics'`
`>`, `<`	Greater / Less	`gpa > 3.5`
`⩾`, `⩽`	Greater/Less or equal	`age ⩽ 21`
`BETWEEN`	In a range	`gpa BETWEEN 3.0 AND 3.8`

🔍 WHERE — Examples 1

```sql
-- Students with GPA above 3.5
SELECT name, gpa
FROM students
WHERE gpa > 3.5;
```

```sql
-- Students in Computer Science OR Mathematics
SELECT name, major
FROM students
WHERE major IN ('Computer Science', 'Mathematics');
```

🔍 WHERE — Examples 2

```sql
-- Combine conditions with AND / OR
SELECT name, age, gpa
FROM students
WHERE age ⩽ 21
  AND gpa ⩾ 3.7;
```

```sql
-- Names that start with the letter 'A' or 'E'
SELECT name FROM students
WHERE name LIKE 'A%' OR name LIKE 'E%';
```

🎯 `%` *is a wildcard — it matches any sequence of characters.*

🔢 ORDER BY — Sorting Results

`ORDER BY` controls the **order** your results come back in.

```
SELECT column1, column2
FROM table_name
ORDER BY column1 ASC;    -- ASC = ascending (default)
                          -- DESC = descending
```

Examples:

```
-- Best students first (highest GPA)
SELECT name, major, gpa
FROM students
ORDER BY gpa DESC;
```

🔢 ORDER BY — More Examples

```sql
-- Alphabetically by major, then by GPA within each major
SELECT name, major, gpa
FROM students
ORDER BY major ASC, gpa DESC;
```

```sql
-- Limit results to top 3
SELECT name, gpa
FROM students
ORDER BY gpa DESC
LIMIT 3;
```

🧩 Putting It Together — The SQL Query Structure

Every SQL query follows this structure (in this exact order!):

```
SELECT    what columns to show              ← Always first
FROM      which table                       ← Where the data lives
WHERE     filter conditions                 ← Optional
ORDER BY  how to sort                       ← Optional
LIMIT     max number of rows to return      ← Optional
```

🧩 Putting It Together — Full Example

```sql
SELECT name, major, gpa
FROM students
WHERE gpa ⩾ 3.5
  AND major ≠ 'Physics'
ORDER BY gpa DESC
LIMIT 5;
```

🧠 ***Memory trick:*** *Think of it as answering: "Show me ___(SELECT) from ___(FROM), but only where ___(WHERE), sorted by ___(ORDER BY)."*

Part 3

Aggregation & Grouping

🔢 Aggregate Functions — Summarizing Data

Instead of seeing every row, sometimes you want **summary statistics**.

Function	What It Does	Example
`COUNT(*)`	Count rows	How many students?
`SUM(col)`	Add up values	Total revenue?
`AVG(col)`	Calculate average	Average GPA?
`MIN(col)`	Find the minimum	Lowest price?
`MAX(col)`	Find the maximum	Highest salary?

🔢 Aggregate Functions — Examples

```sql
-- How many students do we have?
SELECT COUNT(*) AS total_students
FROM students;

-- What's the average GPA?
SELECT AVG(gpa) AS average_gpa
FROM students;

-- What are the min and max GPAs?
SELECT MIN(gpa) AS lowest,
       MAX(gpa) AS highest
FROM students;
```

📊 GROUP BY — Aggregating by Category

`GROUP BY` is where the magic happens. It lets you compute aggregate statistics **for each group** in your data.

```
SELECT group_column, AGGREGATE(value_column)
FROM table_name
GROUP BY group_column;
```

📊 GROUP BY — Example

```sql
-- Average GPA broken down by major
SELECT
    major,
    COUNT(*) AS num_students,
    ROUND(AVG(gpa), 2) AS avg_gpa,
    MAX(gpa) AS top_gpa
FROM students
GROUP BY major
ORDER BY avg_gpa DESC;
```

📊 GROUP BY — Sample Output

major	num_students	avg_gpa	top_gpa
Computer Science	2	3.80	3.9
Mathematics	2	3.60	3.8
Physics	1	3.20	3.2

🎛️ HAVING — Filtering Groups

`WHERE` filters rows **before** grouping. `HAVING` filters **after** grouping.

```sql
SELECT major, COUNT(*) AS num_students, AVG(gpa) AS avg_gpa
FROM students
GROUP BY major
HAVING COUNT(*) ⩾ 2;    -- Only show majors with 2+ students
```

The Order of Clauses

```
SELECT    ...
FROM      ...
WHERE     ...     ← Filters ROWS (before grouping)
GROUP BY  ...
HAVING    ...     ← Filters GROUPS (after grouping)
ORDER BY  ...
LIMIT     ...
```

💡 *Think of `HAVING` as a `WHERE` clause specifically for groups.*

Part 4

Joining Tables

🔗 Why We Need Multiple Tables

In real databases, data is spread across **multiple related tables** to avoid repetition. This is called **normalization**.

Instead of this (bad — repetitive data):

order_id	customer_name	customer_email	product	price
1	Alice	alice@email.com	Laptop	999
2	Alice	alice@email.com	Mouse	25
3	Bob	bob@email.com	Keyboard	75

We use two separate tables (good):

- `customers` (customer_id, name, email)
- `orders` (order_id, customer_id, product, price)

🤝 INNER JOIN — The Most Common Join

`INNER JOIN` returns rows where there is a **match in both tables**.

```
SELECT customers.name, orders.product, orders.price
FROM orders
INNER JOIN customers
    ON orders.customer_id = customers.customer_id;
```

Visualizing the JOIN:

customers table

id	name
1	Alice
2	Bob
3	Carol

JOIN

orders table

id	c_id	product
1	1	Laptop
2	1	Mouse
3	2	Keybd

(Carol has no orders - excluded)

🗂️ Types of JOINs at a Glance

JOIN Type	Returns	Use Case
`INNER JOIN`	Rows matching in **both** tables	Most common — only matched data
`LEFT JOIN`	All left rows + matched right	Keep all left, even without a match
`RIGHT JOIN`	All right rows + matched left	Keep all right, even without a match
`FULL OUTER JOIN`	All rows from both	Everything, matched or not

🗂️ LEFT JOIN Example

```sql
-- LEFT JOIN example:
-- All customers, even if they haven't ordered
--
SELECT customers.name, orders.product
FROM customers
LEFT JOIN orders ON customers.customer_id = orders.customer_id;
```

🔑 *Start with `INNER JOIN`. It covers 80% of real-world cases.*

Part 5

DuckDB in Python

DuckDB + Python — The Setup

```
# Install (run once in terminal)
# pip install "duckdb≥2.0" pandas matplotlib seaborn

import duckdb
import pandas as pd

# Create an in-memory database
con = duckdb.connect()

# Create a table
con.execute("""
    CREATE TABLE students (
        student_id INTEGER,
        name       VARCHAR,
        age        INTEGER,
        major      VARCHAR,
        gpa        DECIMAL(3,2)
    )
""")

# Insert data
con.execute("""
    INSERT INTO students VALUES
    (1, 'Alice Chen',   20, 'Computer Science', 3.9),
    (2, 'Bob Martin',   22, 'Mathematics',      3.4),
    (3, 'Carol White',  21, 'Computer Science', 3.7),
    (4, 'David Lee',    23, 'Physics',          3.2),
    (5, 'Emma Davis',   20, 'Mathematics',      3.8)
""")
```

⚡ Running Queries from Python

```
# Method 1: Fetch as a list of tuples
result = con.execute("SELECT * FROM students").fetchall()

# Method 2: Fetch as a Pandas DataFrame (recommended!)
df = con.execute("""
    SELECT major,
           COUNT(*) AS num_students,
           ROUND(AVG(gpa), 2) AS avg_gpa
    FROM students
    GROUP BY major
    ORDER BY avg_gpa DESC
""").df()

print(df)
#           major  num_students  avg_gpa
# 0  Computer Science             2     3.80
# 1      Mathematics             2     3.60
# 2          Physics             1     3.20
```

🔮 The Superpower: Query CSV Files Directly!

DuckDB can query **raw CSV files** without even loading them first:

```
import duckdb

# Query a CSV file as if it were a database table!
result = duckdb.sql("""
    SELECT department,
           COUNT(*) AS headcount,
           ROUND(AVG(salary), 0) AS avg_salary
    FROM 'employees.csv'
    GROUP BY department
    ORDER BY avg_salary DESC
""").df()
```

```
# Even query multiple CSV files at once!
result = duckdb.sql("""
```

🦆 DuckDB Can Also Query Pandas DataFrames!

```
import duckdb
import pandas as pd

# Create a Pandas DataFrame
df = pd.DataFrame({
    'city':  ['NYC', 'LA', 'Chicago', 'NYC', 'LA'],
    'sales': [1200, 850, 430, 980, 720]
})

# Query it directly with SQL!
result = duckdb.sql("""
    SELECT city,
           COUNT(*) AS transactions,
           SUM(sales) AS total_sales,
           ROUND(AVG(sales), 0) AS avg_sale
    FROM df
    GROUP BY city
    ORDER BY total_sales DESC
""").df()
```

🧪 Hands-On Lab

Time to Get Your Hands Dirty

📋 Lab Overview

Open the Jupyter Notebook: `duckdb_intro_lab.ipynb`

You'll explore 3 real-world datasets:

🛒 **E-Commerce Orders**

- Products, customers, transactions

🎬 **Movies Database**

- Films, genres, ratings, box office

🏛️ **Employee Records**

- Staff, departments, salaries

You'll run queries that:

- ✅ Filter with `WHERE`
- ✅ Sort with `ORDER BY`
- ✅ Aggregate with `GROUP BY`
- ✅ Visualize with `matplotlib`
- ✅ Join two tables with `INNER JOIN`

🎯 ***Goal:*** *By the end of today, you'll write real SQL queries on real data and*

🛠️ SQL Cheat Sheet — Keep This Handy!

```sql
-- Full query template (clauses must be in this order)
SELECT    col1, col2, AGG_FUNC(col3) AS alias      -- What to show
FROM      table_name                               -- Where data is
JOIN      other_table ON table.key = other.key     -- Combine tables
WHERE     condition1 AND condition2                -- Filter rows
GROUP BY  col1, col2                               -- Group for aggregation
HAVING    AGG_FUNC(col3) > value                   -- Filter groups
ORDER BY  col1 DESC, col2 ASC                      -- Sort results
LIMIT     10;                                      -- Cap row count
```

Aggregate Functions

- `COUNT(*)`
- `SUM(col)`
- `AVG(col)`
- `MIN(col)`
- `MAX(col)`
- `ROUND(val, n)`

WHERE Operators

- `=`
- `≠`
- `>`
- `<`
- `≥`
- `≤`
- `BETWEEN`
- `IN (...)`
- `LIKE '%pattern%'`
- `IS NULL`
- `IS NOT NULL`

Boolean Logic

- `AND`
- `OR`
- `NOT`

🎓 Key Takeaways

🦆 DuckDB

- Runs inside Python, zero setup
- Full SQL support
- Reads CSV, JSON, Parquet
- Queries Pandas DataFrames
- Perfect for analytics

🏛️ Database Fundamentals

- Tables = rows + typed columns
- Primary keys = unique identifiers
- Normalization = no repetition
- Relationships via foreign keys

📝 SQL Essentials

- `SELECT` — choose columns
- `FROM` — choose table
- `WHERE` — filter rows
- `ORDER BY` — sort output
- `LIMIT` — cap results
- `GROUP BY` — aggregate
- `HAVING` — filter groups
- `JOIN` — combine tables

🚀 What's Coming Next

Week	Topic
Week 1	✅ **Today** — DuckDB, SELECT, WHERE, GROUP BY
Week 2	Advanced JOINs, Subqueries, CTEs
Week 3	Window Functions (RANK, LEAD, LAG, PARTITION BY)
Week 4	Data Modeling & Schema Design
Week 5	Indexing, Query Optimization, EXPLAIN
Week 6	Project — Build a Real Analytical Dashboard

> 🌟 *SQL is one of the **most in-demand skills** in tech, data science, finance, and business. You're making a great investment today.*



You're Ready.

Open your Jupyter Notebook and run your first SQL query!

```sql
SELECT 'Hello, World of Data!' AS message;
```

Questions? Raise your hand — no question is too basic.

Database Management Systems 101 · Day 1

📚 Resources & Further Reading

Official Documentation

- 🦆 DuckDB Docs: `duckdb.org/docs`
- 📘 SQL Tutorial: `sqlzoo.net`
- 🎓 Mode SQL Tutorial: `mode.com/sql-tutorial`

Practice Platforms

- `leetcode.com` — SQL challenges
- `kaggle.com` — Real datasets + notebooks
- `sqlpractice.io` — Interactive SQL exercises

Books

- **Learning SQL** — Alan Beaulieu (O'Reilly)